ROCKETRIDE EXPLAINED / 01

# Redis: the technical picture

A shared data service you can use from Python. Start with its architecture, then look at the code on page 2.

**Redis is an in-memory data structure server.** Your Python application sends commands to a separate Redis process, usually over a TCP connection. Redis executes the commands against data in its own RAM and sends results back. The server can run on your laptop or another machine. [4, 5]

## How this differs from a Python dictionary

| Python dict | Redis |
|---|---|
| An object inside your Python process. | A service running outside your Python process. |
| Other app processes do not automatically share it. | Multiple processes or machines can access the same data. |
| Its contents disappear when that process ends. | Data can outlive your app process. Surviving a Redis restart depends on persistence settings. |
| Expiration needs application code. | Keys can have a TTL, a built-in expiration time. |

Redis is useful for **shared state**: cached results, login sessions, or counters used by several app workers. It adds a connection and possible network failures, so it is not a faster replacement for every local dictionary.

## What it stores

Each key names a value with a Redis data type. **Strings** can hold text, bytes, or JSON encoded as text. **Hashes** hold named fields; **lists** hold sequences; **sets** hold unique members. Arbitrary Python objects are not stored directly. [4]

## RAM and disk serve different purposes

For a typical standalone Redis server, RAM holds the working dataset. Optional **persistence** writes recovery data to disk: **RDB** takes snapshots, while **AOF** logs writes that can be replayed after restart. What survives a crash depends on these settings; an acknowledged write is not an unconditional guarantee against data loss. [6]

A cache can often regenerate lost entries. Session or application data may need stronger durability. That is a design choice to confirm, not something our connector automatically provides.

ROCKETRIDE EXPLAINED / 02

# A Redis cache in Python

The redis-py package is a client library. Installing it does not start a Redis server; a server must already be running.

This example caches **one fixed question**. The answer string stands in for a model response. The real app derives its key from the request and context. [5]

```
import redis
r = redis.Redis(host="localhost", port=6379,
                decode_responses=True)
key = "aic:answers:what-is-redis:v1"
answer = r.get(key)
if answer is None:
    # Real app: call the model for this question.
    answer = "Redis is an in-memory data structure server."
    r.set(key, answer, ex=300)
    source = "generated"
else:
    source = "cache"

print(source, answer)
r.close()
```

## What happens on each call

**r.get(key)** sends GET to the server. With decode_responses=True, a stored text value returns as a Python str; a missing key returns None. Checking "is None" also handles a saved empty string correctly.

**r.set(..., ex=300)** sends SET with a 300-second TTL. It stores the value and expiration in one command. Retrieving a saved value is a **cache hit**; no saved value means a **cache miss**. Expiry makes the entry unavailable to later reads. [5, 7]

## Why the server matters for counters

**INCR** atomically increases an integer on the server, so concurrent increments do not overwrite each other. Combining separate commands, such as incrementing and setting expiry, still needs deliberate design. [8]

**Your RocketRide node wraps this client behavior:** receive an operation and inputs from RocketRide, call Redis, and return a structured result or error. It also enforces the agreed connection settings and access limits.

ROCKETRIDE EXPLAINED / 03

# What are we building for RocketRide?

There are two main technical pieces: the Redis connector and an app that demonstrates it.

**RocketRide lets developers connect reusable components to build an AI workflow.** A workflow is a sequence of actions, such as receiving a question, checking saved information, generating an answer, and returning it.

RocketRide calls these components **nodes**. It calls the connected workflow a **pipeline**. Here, a node simply means a component that performs a job.

| Piece | Its job in our example |
|---|---|
| The app | The screen where a person asks a question and reads the answer. |
| The Redis node | The component that lets RocketRide send save, read, and other agreed requests to Redis. |
| Redis itself | The separate software that holds the saved information. |

## Your team's first piece: the Redis node

You write the connection between RocketRide and Redis. For example, RocketRide gives your component a label and an answer. Your component saves that answer in Redis and reports whether it worked.

For a read request, your component receives a label and returns the saved answer, or reports that no entry exists. If Redis cannot be reached, it returns an understandable error.

## Your team's second piece: the demonstration app

You build an app inside RocketRide that uses your component. RocketRide suggests Answer Cache, the app on page 2. Their **App Builder** is the feature in the VS Code extension used to create and preview the app.

**A sentence you can use in the meeting:**
"We are building a component that lets RocketRide save and retrieve information using Redis, plus an app that shows the component working."

The full engagement also includes tests, documentation, a live demo, a blog, and community posts. The client brief describes who helps publish and record those materials. [1]

ROCKETRIDE EXPLAINED / 04

# What should the Redis node do?

This is a first proposal to discuss with RocketRide. The meeting should turn it into an agreed feature list.

**Scope** means deciding exactly what your team will deliver. These proposed actions support Answer Cache and keep the node useful for other apps too.

| Proposed action | Why an app might need it |
|---|---|
| Save and read an entry | Store an answer and retrieve it when the question comes back. |
| Delete an entry | Remove an answer that should no longer be used. |
| Set or check expiration | Give an answer a time limit and find out how long it has left. |
| Increase a counter | Track a number, such as how many questions a user has asked. |
| Find related entries in batches | Find saved answers for one topic so an authorized user can clear that topic. |

## The node also needs connection settings

It needs to know where Redis is running and whether a username or password is required. RocketRide should show you how their platform stores these settings securely.

## The brief already sets some boundaries

**Limit which entries it can touch.** The brief requires a configured prefix, a shared beginning on the labels. For example, all allowed labels could begin with "aic:". The node must stay within its assigned group.

**Reading is the default.** Saving, deleting, and other changes are allowed only after a setting enables them. This is what "read-only unless writes are enabled" means.

**Keep actions limited.** The brief excludes commands that clear a whole database, change server settings, or run unrestricted scripts. Finding and deleting entries should happen in controlled amounts. [1]

**Must-have versus stretch:** must-have features are required to finish the agreed project. Stretch features are extras if time allows. Discuss the basic actions above first. More advanced matching and real-time event features need a separate decision.

ROCKETRIDE EXPLAINED / 05

# Which app should we propose?

Bring three ideas to discuss. Your team is choosing one demonstration app, not promising to build three.

| App idea | What someone would use it for |
|---|---|
| 1. Answer Cache | Ask questions and reuse saved AI answers. This is the app RocketRide proposes in its brief. |
| 2. Session Follow-up Assistant | Save selected facts from a conversation so a later workflow run can retrieve them. Ask how this differs from RocketRide's existing memory feature. |
| 3. Temporary Result Pickup | Save a workflow result, give someone a lookup code, and let them retrieve it before it expires. |

## Start the discussion with Answer Cache

The screen could contain a question box, an answer, and a label showing whether the answer was newly generated or retrieved from Redis.

RocketRide's proposed app also includes these features:

**Recognize repeated questions:** decide which wording differences should still reuse an answer.
**Limit questions per user:** use a counter so one person cannot ask unlimited questions in a minute.
**Clear a topic:** let an authorized administrator remove outdated answers in one category.
**Show results:** display how often saved answers were reused and estimate the model cost avoided. [1]

## One question that really matters

Should "What is Redis?" and "Explain Redis to me" share an answer? Changing capitalization is simpler than recognizing equivalent meaning. The brief mentions both, so ask them to give examples of what the first version must recognize.

**Suggested position:**
"Answer Cache looks like a good demonstration. Can we confirm which of its features are required first, especially recognizing differently worded questions?"

The app decides when to reuse an answer. The Redis node saves and retrieves it.

ROCKETRIDE EXPLAINED / 06

# How the team gets it done

Personal role assignments remain blank. Use the role descriptions to divide the work together.

| Role | What that person owns | Name |
|---|---|---|
| Node Lead | Design and build the Redis component; handle code review feedback. | _____ |
| Integration & Test Lead | Help everyone run the project and check that its parts work together. | _____ |
| App Lead | Build the demonstration app and run the live demo. | _____ |
| Content Lead | Coordinate written explanations, posts, meeting logistics, and demo materials. | _____ |

## The dates in RocketRide's proposal

| Date | What should be ready |
|---|---|
| October 30 | An agreed design and testing plan. |
| November 6 | Node, tests, and instructions submitted for review as a pull request (PR). |
| November 13 | Review fixes complete and code accepted (merged) into their repository. |
| November 20 | Node released for others to use; app built and tested. |
| November 25 | Live demo and recording. Blog and community posts follow the release. |

The brief also schedules social posts for early December. It expects weekly RocketRide engineering calls and separate AIC Board check-ins. Confirm the dates and owners on the call. [1]

**Why release timing matters:** the brief says the hosted app must use a released node. If release is delayed, its proposed fallback is to demonstrate the app in App Builder preview and wait to publish the release-linked posts.

ROCKETRIDE EXPLAINED / 07

# What to say in the meeting

You can ask for a walkthrough. You do not need to arrive knowing every part of RocketRide's code.

## Open with this

> "We understand that we are building a component that lets RocketRide save and retrieve information using Redis. Then we will build an app showing it working, potentially by saving and reusing AI answers. Could you show us an existing workflow and where our component should fit?"

## Then ask these questions

**1. What must our first version do?**
Show the proposed actions on page 4. Ask which are required and which can wait.

**2. Are we building Answer Cache?**
Ask which app features on page 5 are essential. Bring the other two ideas as alternatives.

**3. What should count as the same question?**
Ask for examples of differently worded questions that should reuse an answer, and examples that should not.

**4. Can you show us a working example node?**
Ask how to run it, how it receives requests, and how it returns results. The Notion node is the reference suggested in Discord.

**5. How do we prove our project is finished?**
Ask what the final demo must show and which tests they expect.

**6. Who helps with setup, review, and release?**
Confirm the right contact, who approves the design, and whether the proposed dates still work.

## Read back what you agreed

"We will build _____. Our app will be _____. The required first-version features are _____. We will leave _____ as stretch. Our next deadline is _____, and _____ will review our design."

The PDF is their starting proposal. The later Discord message asks you to bring a rough scope and app ideas that you can refine together. Record the decisions rather than assuming every proposed detail is already settled. [1, 2]

ROCKETRIDE EXPLAINED / 08

# How to use the files in VS Code

Read the guide first. Then inspect one source file at a time with a specific question in mind.

| Open file | The question it helps answer |
|---|---|
| Original client PDF | What did RocketRide propose that we deliver? |
| services.json | How does a node describe itself and the settings a user can choose? |
| IGlobal.py | How does the node read its settings and prepare to run? |
| IInstance.py | What actions can this node perform, and what does each return? |
| test_notion.py | How do developers automatically check whether the existing Notion node behaves correctly? |
| app-builder.md | How do we create, preview, and publish the demonstration app? |

## What has been checked so far

During preparation, 78 automated Notion behavior checks and 2 checks of its node description passed. A test is a program that checks an expected result. These results help us understand the example; they do not prove a new Redis node works.

The actual Redis node has not been built. Running Redis on this computer and trying the complete app are still pending. Docker, a tool used to run Redis locally in a packaged environment, was not available. The VS Code workspace is currently open for source browsing.

Your earlier **rocket-ride** folder holds the separate GitHub issue-connector exercise. The **rocketride-server** folder is the actual RocketRide source used for this review. [3]

## Where the information comes from

[1] Your supplied RocketRide project PDF, version 2, pages 1-7.
[2] Your Discord screenshot, showing the October 27 preparation request.

[3] Inspected RocketRide source; Notion example; node overview; App Builder guide.

[4] Redis data store guide; [5] Python client guide; [6] persistence; [7] SET and expiration; [8] INCR and counters.

The original client brief is unchanged. Personal role assignments remain blank.